NetSage AI

AI-Assisted Cisco Network Troubleshooting and Diagnostic System

STUDENT NAME

Utkarsh Kumar

ENROLLMENT NUMBER

CS-2341388

PROGRAM & SEMESTER

B.Tech CSE, 7th Semester

ORGANIZATION NAME

Cisco Networking Academy

PROJECT OVERVIEW

Introduction

- Cisco networking labs are unforgiving: a single misconfigured VLAN, gateway, DHCP scope, ACL, or NAT rule can break connectivity.
- Students and junior engineers often know individual commands but struggle to connect a symptom to its root cause.
- NetSage AI reads a symptom plus show-command evidence, identifies a likely cause using a rule-based diagnostic engine, reports the OSI layer, recommends a verification command, and proposes safe fix steps.
- The diagnosis remains advisory — it is never allowed to directly apply a configuration change.

The implementation combines:

30 structured troubleshooting cases • a Python diagnostic engine • a deterministic checker • an interactive HTML/CSS/JS dashboard • a structured diagnosis prompt • a Responsible AI review workflow

THE CHALLENGE

Problem Statement

- A learner may observe a PC can't reach a server, a VLAN doesn't cross a trunk, DHCP clients get wrong addresses, DNS resolution fails, or an ACL blocks traffic.
- The challenge is to identify a likely root cause and the next verification step — without inventing evidence or applying an unsafe configuration change.

The specification requires the system to:

- Accept symptoms, Packet Tracer notes, and show-command outputs
- Recommend a likely fault and OSI layer, with supporting evidence
- Suggest a next diagnostic command and evidence-backed fix steps
- Require human review before any configuration change is accepted

GOALS

Project Objectives

- Develop an AI-assisted rule-based diagnostic engine for common Cisco network faults
- Prepare a structured set of 30 troubleshooting cases with symptoms, evidence, faults, OSI layer & severity
- Create a structured diagnosis format: root cause, confidence, evidence, next command, fix steps
- Develop a deterministic Python checker that independently validates common configuration clues
- Build an interactive dashboard for case metrics, issue types, severity & Responsible AI details
- Require human review before any suggested configuration change is accepted
- Record accepted and corrected responses so diagnosis quality can be inspected and audited
- Evaluate the implemented rule-based diagnosis against the known expected faults in the dataset

BOUNDARIES

Scope of the Project

Focused on Cisco Packet Tracer / lab-style structured evidence — an academic learning and diagnostic-support tool, not an autonomous network manager.

In Scope

- VLAN & trunk configuration problems
- Gateway, subnet mask & IP addressing
- DHCP and DHCP relay problems
- DNS server and stale-record issues
- Routing, static routes, OSPF, default routes
- ACL and application-access problems
- NAT/PAT translation & interface issues
- Wireless VLAN & guest-isolation scenarios
- Interface state, speed/duplex, SVI faults
- Responsible AI review records

Out of Scope

- Direct connection to live production routers or switches
- Automatic configuration deployment to network devices
- Automatic learning of new rules from live traffic
- A claim of real-world diagnostic accuracy beyond the supplied dataset

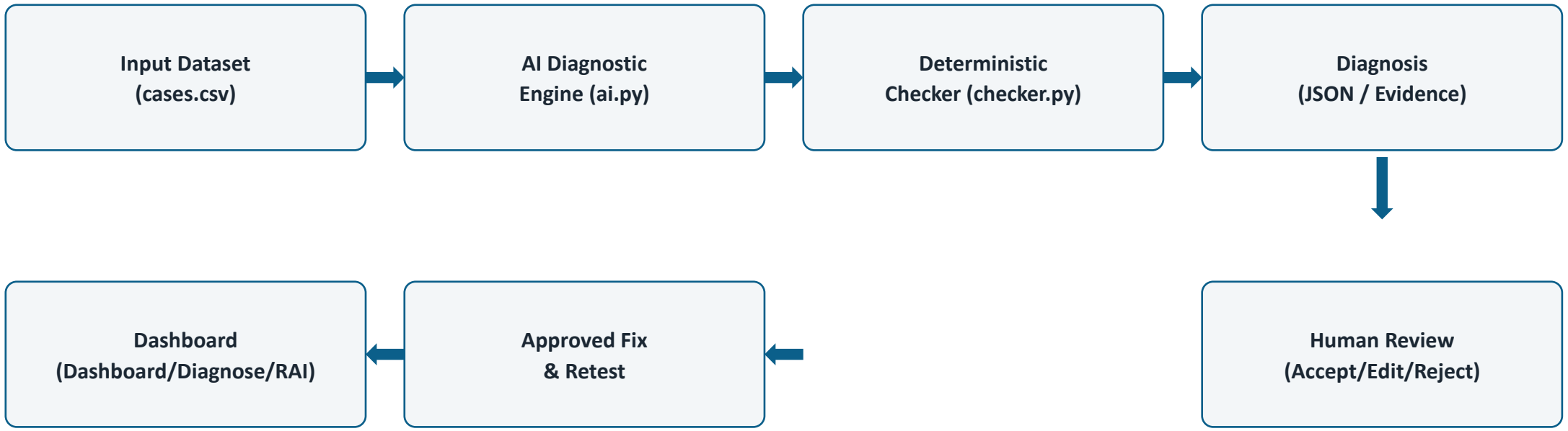
TECH STACK

Technologies and Tools Used

Technology	Use	Purpose
Python	AI diagnostic engine & deterministic checker	Rule-based diagnosis, evidence matching, validation
CSV	Case dataset & Responsible AI review log	Structured storage of cases and human-review records
HTML / CSS / JS	Interactive dashboard	Case exploration, diagnosis display, review workflow
Cisco Packet Tracer	Laboratory environment	Cisco-style scenarios and command evidence
Markdown	Structured prompt library	Prompt instructions, schema & worked examples

HOW IT WORKS

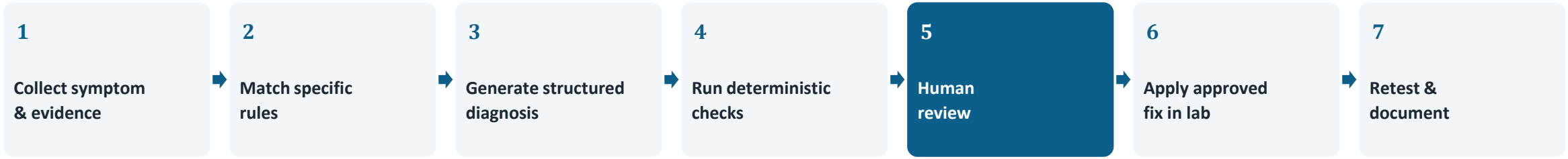
System Architecture



Configuration changes are never applied automatically — every diagnosis passes through human review first.

EVIDENCE-TO-DECISION WORKFLOW

Methodology



Rule-based diagnosis:

Specific patterns (e.g. "vlan 30 absent", "no ip helper-address", "deny tcp") are matched before generic fallbacks, so OSPF and other special cases get precise diagnoses rather than a vague catch-all.

Structured output:

root_cause · confidence · OSI layer · evidence · next_command · fix_steps · human_review: required — only supplied evidence is used; insufficient evidence yields a low-confidence response.

Standard fix sequence: verify the finding with the suggested command → apply the approved configuration change → retest the original symptom.

RESULTS

Dataset Coverage & Evaluation

30 / 30

cases matched the expected fault (100%) at dataset level; all received high confidence. This is a dataset-level rule-based evaluation, not a claim of real-world diagnostic accuracy.

Severity: 17 High · 12 Medium · 1 Critical

OSI layers: 15 Layer 3 · 5 Layer 2 (+ Layer 4, 7 & combined)

Category	Cases	Severity
ACL	4	High/Medium
VLAN	3	High/Medium
Routing	3	High/Medium
Wireless	3	Critical/High/Med
DHCP	2	High/Medium
DNS	2	Medium
NAT	2	High
Interface	2	High/Medium
Other categories	9	Mixed

INTERFACE

Dashboard Modules

30	18	17	50%
Total Cases	High/Critical	Issue Types	AI–Human Agreement

Dashboard

Case metrics, issue types, severity and OSI distribution.

Diagnose

Case selection, root cause, confidence, evidence and commands.

Responsible AI

Review policy and accepted/edited/rejected records.

All three sections carry the same note: human review is required before any suggested change is acted on.

RESPONSIBLE AI

Human-in-the-Loop Review

10 reviewed cases logged — 5 accepted as-is, 5 edited after human verification. No configuration change is ever applied without approval.

Case	Type	Reviewer Correction
C006	ACL	Confirmed ACL blocks HTTP specifically; verify direction before changing
C015	OSPF	Verify OSPF network statement and area configuration before changing
C020	NAT	Inspect NAT rule and inside/outside interface definitions before modifying
C025	ACL	Verify ACL entry, interface direction, and HTTPS requirement before editing
C030	ACL	Confirm ACL applied to management interface in correct direction

5 of 10 reviewed diagnoses required no correction (50% AI–human agreement); the other 5 needed direction or context checks before a fix was trusted.

WRAPPING UP

Limitations, Future Scope & Conclusion

Current Limitations

- Manually defined rules — no live-device connection
- Does not automatically learn new rules from traffic
- 100% match reflects the supplied fixed dataset only

Future Scope

- Integrate an LLM for free-form CLI understanding
- Secure live SSH / NETCONF / RESTCONF integration
- Expand to SD-WAN, security & multi-vendor scenarios
- NLP/ML classifier, authentication & ticketing integration

Conclusion

NetSage AI demonstrates how AI-assisted, rule-based reasoning can support network troubleshooting in an academic Cisco lab environment. It combines structured cases, evidence-based diagnosis, deterministic validation, an interactive dashboard, and mandatory human review into one integrated solution.

The project provided practical experience in networking, Artificial Intelligence, Python, data organization, web interface development, testing, and responsible technology use.